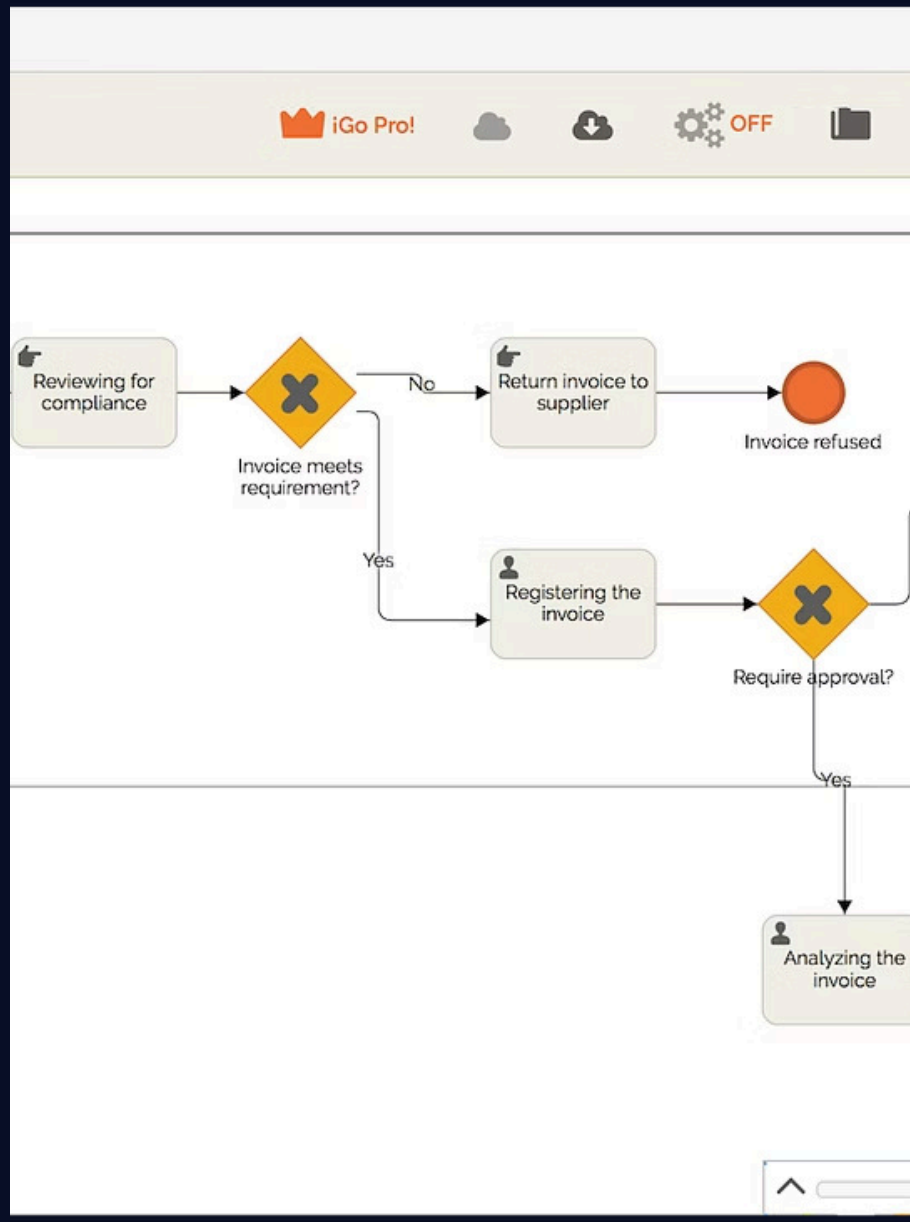
Workflow Orchestration para Data Mining & Engineering

De tareas sueltas a procesos confiables

Un recorrido por las técnicas, herramientas y mejores prácticas para transformar tareas analíticas ad-hoc en flujos de trabajo robustos, predecibles y automatizados.



Objetivos del curso

Comprender los fundamentos

Entender qué es la orquestación de workflows, su vocabulario esencial y por qué es crítica para proyectos de datos exitosos.

Diseñar flujos robustos

Aprender a crear DAGs efectivos, manejar fallos y establecer sistemas observables para distintos casos de uso.

Implementar soluciones prácticas

Conocer las herramientas disponibles, patrones de arquitectura y aplicarlos a casos reales de data mining y engineering.

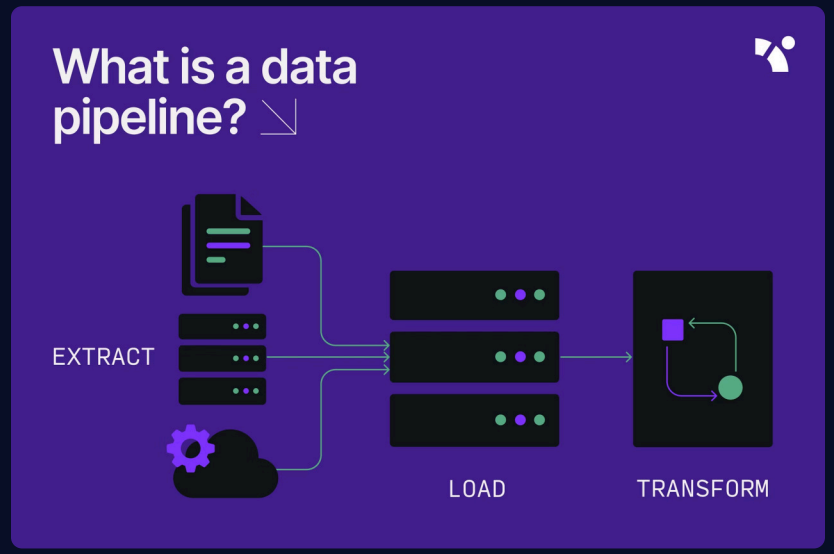
Al finalizar, serás capaz de transformar procesos analíticos sueltos en pipelines confiables, ya sea en entornos batch, micro-batch, streaming o event-driven.

¿Qué es orquestación de workflows?

La orquestación es el proceso de **coordinar, integrar y gestionar** múltiples tareas computacionales y sus dependencias dentro de un flujo de trabajo.

No se trata solo de automatizar tareas individuales, sino de crear un sistema cohesivo donde:

- Se define **cuándo**, **dónde** y **cómo** se ejecuta cada tarea
- Se gestiona el **estado**, **reintentos** y **manejo de fallos**
- Se proporciona **observabilidad** mediante logs, métricas y alertas



*El objetivo principal: procesos **predecibles y repetibles*** que liberen tiempo para el análisis.

Beneficios en Data Mining

Reproducibilidad

Pipelines consistentes para EDA, ingeniería de features y entrenamiento de modelos que documentan cada paso del proceso.

Trazabilidad

Seguimiento completo desde los datos de origen hasta los resultados finales, facilitando auditorías y comprensión del proceso.

Automatización

Ejecución programada de procesos recurrentes sin intervención manual, reduciendo errores humanos.

Foco analítico

Menos tiempo dedicado a tareas operativas y más al análisis de valor, reduciendo la deuda técnica.

La orquestación conecta perfectamente con metodologías como CRISP-DM, permitiendo que las fases de preparación de datos, modelado y despliegue sean procesos estructurados y no tareas improvisadas.

Glosario esencial de orquestación

DAG (Directed Acyclic Graph)

Estructura que define tareas y sus dependencias sin ciclos. Cada nodo es una tarea y cada arista representa una dependencia.

Task / Operator

Unidad atómica de trabajo que realiza una acción específica (ej: consulta SQL, transformación de datos, entrenamiento de modelo).

Scheduler / Executor

El scheduler planifica cuándo deben ejecutarse las tareas. El executor se encarga de la ejecución efectiva de dichas tareas.

Sensors / Triggers

Mecanismos que esperan eventos o condiciones específicas para iniciar la ejecución de un flujo de trabajo.

SLAs (Service Level Agreements)

Tiempos objetivo para la finalización de tareas o flujos completos, con alertas si se exceden.

Modos de orquestación

Batch

Procesamiento de grandes volúmenes de datos en intervalos definidos (diarios/horarios). Eficiente en recursos pero con mayor latencia.

Micro-batch

Procesamiento en intervalos cortos (minutos) de pequeños lotes de datos. Balance entre eficiencia y latencia.

Streaming

Procesamiento continuo de datos en tiempo (casi) real. Menor latencia pero mayor complejidad y coste.

Event-driven

Ejecución en respuesta a eventos específicos (llegada de archivos, mensajes, llamadas API). Reactivo y eficiente.

Trade-offs principales

Modo	Latencia	Costo	Complejidad
Batch	Alta	Bajo	Baja
Micro-batch	Media	Medio	Media
Streaming	Mínima	Alto	Alta
Event-driven	Baja	Medio	Media

Los sistemas **híbridos** combinan diferentes modos según el caso de uso, optimizando recursos.

Diseño de DAGs efectivos

Granularidad correcta

Tareas ni demasiado pequeñas (overhead) ni demasiado grandes (difíciles de gestionar). Cada tarea debe tener una responsabilidad única y clara.

Idempotencia y determinismo

Ejecutar la misma tarea múltiples veces con los mismos inputs debe producir siempre el mismo resultado, sin efectos secundarios.

Fan-in / Fan-out controlado

Evitar demasiadas dependencias entrantes (fan-in) o salientes (fan-out) que dificulten el mantenimiento y la depuración.

Reintentos con backoff

Implementar políticas de reintento inteligentes con espera incremental entre intentos para manejar fallos transitorios.

Timeouts y recursos limitados

Establecer límites claros de tiempo y recursos para evitar tareas zombies o consumo excesivo.

Los mejores DAGs son **pequeños, testeables y recomponibles**, siguiendo los principios de ingeniería de software.

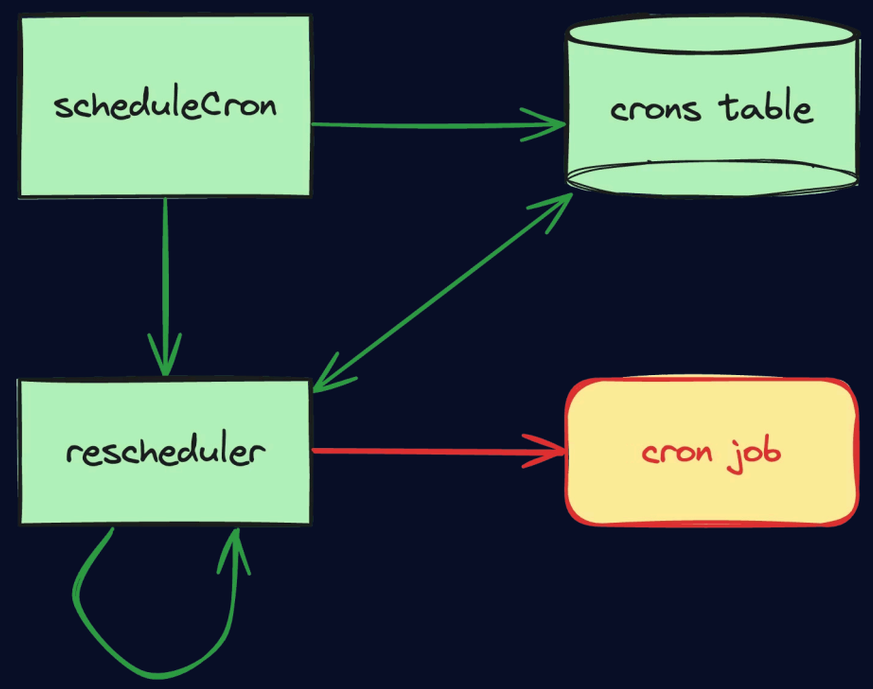
Scheduling y disparadores

Programación basada en tiempo

- **Cron expressions**: Define patrones precisos de ejecución (diario, hora específica, días laborables, etc.)
- **Intervalos**: Ejecución periódica cada X tiempo
- **Catchup**: Recuperación automática de ejecuciones perdidas
- **Max_active_runs**: Control de concurrencia para evitar sobrecarga

Programación basada en datos/eventos

- **Data-aware scheduling**: Ejecución cuando datos están disponibles
- **File/object triggers**: Al detectar nuevos archivos en S3/GCS
- **Message triggers**: Respuesta a mensajes en colas/topics
- **API triggers**: Iniciación vía llamadas REST



Evitar overlapping no deseado

Un error común es programar tareas sin considerar su duración, provocando que una nueva ejecución comience antes de que finalice la anterior.

La gestión de **prioridades y pools de ejecución** permite asignar recursos adecuadamente entre flujos críticos y secundarios.

Estados y manejo de fallas

Success

Tarea completada correctamente. Permite que las tareas dependientes comiencen.

Failed

Error en la ejecución. Activa políticas de reintento y, eventualmente, alertas.

Upstream Failed

No ejecutada porque una dependencia previa falló, evitando procesamiento innecesario.

Skipped

Tarea omitida intencionalmente según condiciones o configuración.

Estrategias de recuperación

- **Retries + backoff exponencial**: Reintentos con tiempos crecientes
- **Dead-letter**: Captura de datos problemáticos para análisis posterior
- **Compensación**: Acciones para revertir efectos parciales
- **Circuit breaker**: Parada automática ante fallos persistentes

Políticas de fallos

Fail-fast: Detener el proceso ante el primer error para evitar propagación de problemas. Ideal para pipelines críticos donde la precisión es esencial.

Resiliente: Continuar procesando a pesar de errores en algunas tareas. Adecuado cuando es aceptable tener resultados parciales.

Backfills y procesamiento de datos históricos

Conceptos clave

Backfill: Proceso de recomputar ventanas de tiempo pasadas, ya sea por corrección de bugs, cambios en lógica de negocio o recuperación de datos perdidos.

Catchup: Configuración que determina si un DAG debe ejecutarse automáticamente para todas las ejecuciones pasadas pendientes.

Watermarks: Marcadores que indican hasta qué punto en el tiempo se han procesado correctamente los datos, garantizando que no queden "huecos".

Estrategias efectivas

- Diseñar para **idempotencia** desde el principio
- Implementar **particionamiento temporal** de datos
- Mantener **control de versiones** tanto de código como de artefactos de datos
- Establecer **mecanismos de bloqueo** para evitar procesamientos simultáneos de la misma partición
- Priorizar los backfills para minimizar impacto en procesamiento regular

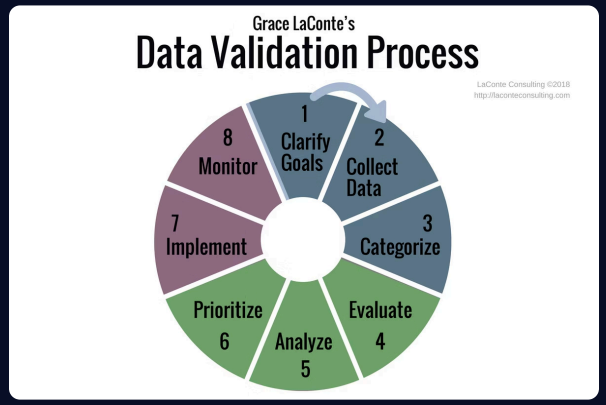
Calidad y contratos de datos

Validación preventiva

- Asserts y tests integrados en tareas
- Contracts/expectations sobre schema y valores
- Verificación antes de promocionar a zonas "gold"

Manejo de anomalías

- Cuarentena para datos sospechosos
- Circuit breakers ante degradación severa
- Alerta temprana sobre desviaciones



> No existe un pipeline confiable sin calidad de datos integrada. La detección temprana de problemas evita la propagación de errores y reduce el coste de corrección.

Las alertas deben ser **accionables** y evitar el ruido. Cada alerta debe indicar claramente:

- Qué falló exactamente
- Por qué es importante
- Qué acciones deben tomarse

Observabilidad y linaje de datos

Pilares de la observabilidad

Logs estructurados

Registros en formato consistente y searchable (JSON) que capturan eventos, errores y métricas operativas.

Métricas

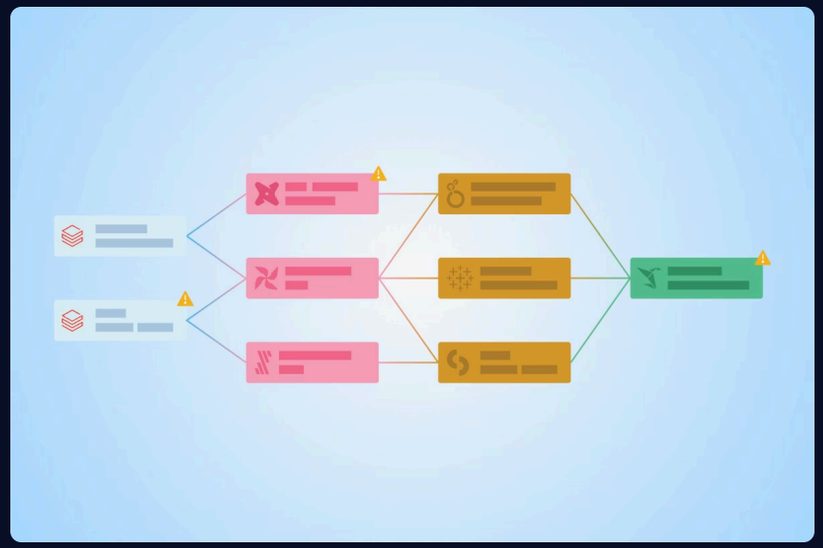
Indicadores cuantitativos sobre latencia, tasas de éxito, volumen de datos y utilización de recursos.

Traces

Seguimiento de una ejecución completa a través de múltiples sistemas, permitiendo identificar cuellos de botella.

Linaje de datos

Documentación de relaciones entre datasets, mostrando cómo se derivan unos de otros y el impacto de cambios.



Un sistema observable permite:

- Detectar problemas **antes** que los usuarios
- Identificar **causas raíz** rápidamente
- Comprender **impactos** de fallos o cambios
- Implementar **mejora continua** basada en datos

Patrones de ingestión y carga de datos

Full Refresh

Reemplaza completamente los datos destino en cada ejecución. Simple pero ineficiente para grandes volúmenes.

Incremental

Procesa solo los datos nuevos o modificados desde la última ejecución, usando timestamps o contadores.

CDC

Captura cambios a nivel de transacción desde la fuente, permitiendo replicar inserciones, actualizaciones y eliminaciones.

Técnicas de carga

- **Upserts/Merge**: Actualiza registros existentes e inserta nuevos en una sola operación
- **SCD (Slowly Changing Dimensions)**: Gestiona histórico de cambios en dimensiones
- **Deduplicación**: Elimina registros duplicados según reglas de negocio
- **Ventanas temporales**: Agrupa datos por períodos para procesamiento eficiente

Optimizaciones clave

La **idempotencia** en sinks (destinos) garantiza que ejecuciones repetidas no generen datos duplicados ni inconsistencias, permitiendo reintentos seguros.

El particionamiento adecuado de datos en Data Warehouses y Data Lakes permite:

- Cargas paralelas eficientes
- Pruning de particiones en queries
- Gestión de ciclo de vida por antigüedad

Seguridad y cumplimiento

Gestión de credenciales

Secret management mediante vaults centralizados (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) que:

- Almacenan credenciales de forma cifrada
- Proporcionan rotación automática
- Registran accesos para auditoría
- Integran con sistemas de identidad corporativos

Control de acceso

RBAC (Role-Based Access Control) que implementa:

- Principio de mínimo privilegio
- Separación de responsabilidades
- Restricción por entorno (dev/prod)

Aislamiento y seguridad

- **Contenedores**: Ejecución en entornos aislados y reproducibles
- **Kubernetes**: Orquestación con políticas de seguridad network
- **Cifrado**: Datos en reposo y en tránsito siempre protegidos
- **Auditoría**: Registro inmutable de todas las ejecuciones y accesos

El tratamiento de **datos sensibles** debe seguir políticas específicas como:

- Enmascaramiento de PII
- Tokenización de datos financieros
- Restricción geográfica según normativas (GDPR, CCPA)

Infraestructura como código

1

Control de versiones

Pipelines y configuración en Git con protección de ramas y revisión de código. Incluye tanto definiciones de DAGs como infraestructura subyacente.

2

CI/CD automatizado

Integración y despliegue continuo que valida sintaxis, ejecuta tests unitarios/integración y despliega a entornos secuenciales.

3

Entornos separados

Segregación clara entre desarrollo, staging y producción con promoción controlada y validación incremental.

4

Testing automatizado

Pruebas que verifican funcionalidad individual de tareas, interacción entre componentes y comportamiento end-to-end.

5

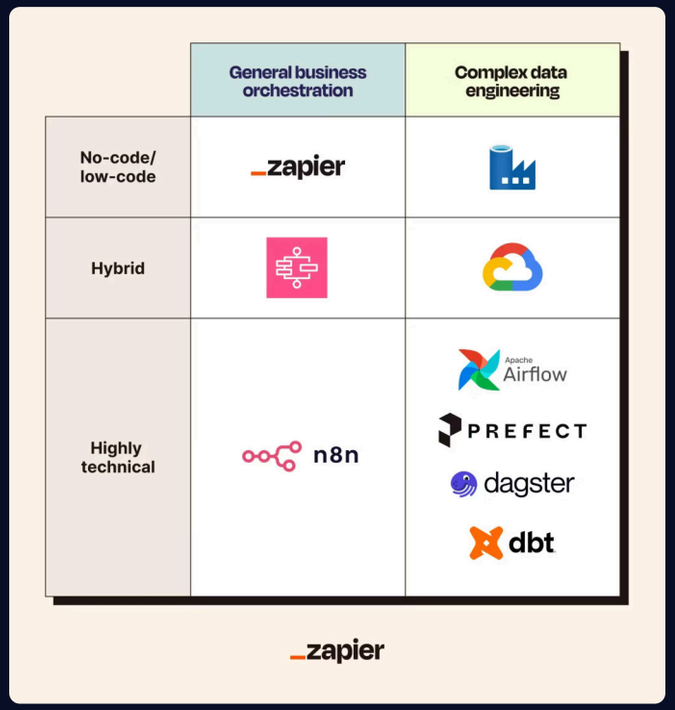
Despliegues controlados

Estrategias como canary releases, blue/green deployments y rollbacks automatizados para minimizar impacto.

La adopción de prácticas DevOps significa **tratar los pipelines como software profesional**, no como scripts ad-hoc, aplicando los mismos estándares de calidad y metodologías ágiles.

Comparativa de herramientas de orquestación

Herramienta	Fortalezas	Casos de uso
Airflow	Maduro, amplio ecosistema, extensible	Batch, gran comunidad, procesos complejos
Prefect	API Pythonica, UI moderna, cloud-ready	Equipos Python-centric, flujos dinámicos
Dagster	Orientado a assets, type checking, testing	Ingeniería de datos moderna, enfoque funcional
Kestra	YAML declarativo, basado en eventos, UI potente	Integración cloud-native, eventos complejos
Otros	Luigi (Spotify), Argo (K8s), Mage, dbt-schedules	Casos específicos por plataforma/tecnología



> No existe la herramienta "mejor", existe la herramienta "**mejor para tu caso**". La elección debe considerar:

- Competencias del equipo
- Infraestructura existente
- Tipos de cargas de trabajo (batch vs eventos)
- Requisitos de observabilidad
- Necesidades de escalabilidad
- Madurez y soporte de la comunidad

Arquitectura de referencia

1

Capa de orquestación

- Scheduler centralizado (control plane)
- Executors distribuidos (data plane)
- Gestión de metadatos y estados

2

Capa de procesamiento

- SQL: Snowflake, BigQuery, Redshift
- Transformaciones: dbt, Spark, Flink
- Python: pandas, scikit-learn, PyTorch

3

Capa de mensajería (opcional)

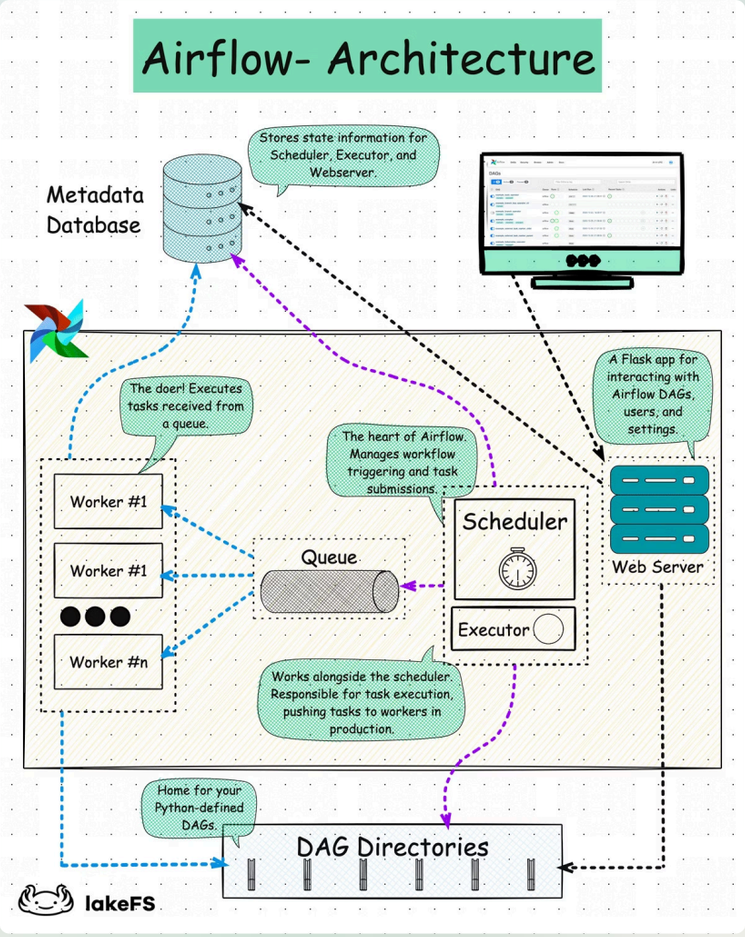
- Message bus: Kafka, Pub/Sub, EventBridge
- Desacoplamiento entre productores/consumidores
- Buffer para picos de carga

4

Capa de almacenamiento

- Data Lake: zonas bronze (raw), silver (validated)
- Data Warehouse: zona gold (business-ready)
- Feature Store: para ML features reutilizables

La **observabilidad centralizada** debe atravesar todas las capas, proporcionando visibilidad end-to-end desde la ingestión hasta el consumo final.



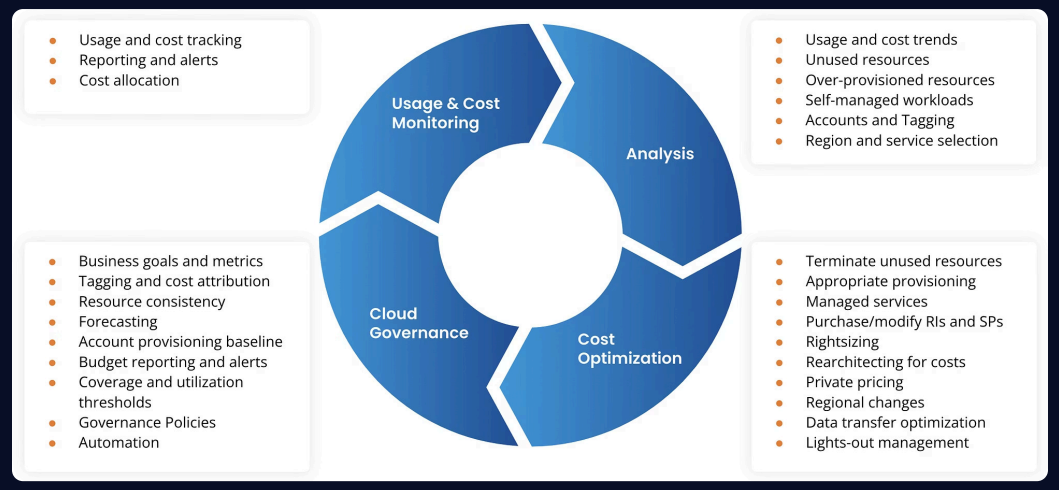
Eficiencia y optimización de costos

Estrategias de paralelismo

- **Paralelismo**: Ejecución simultánea de múltiples tareas independientes
- **Concurrencia**: Número máximo de ejecuciones activas del mismo DAG
- **Autoscaling**: Ajuste dinámico de recursos según demanda
- **Colas de prioridad**: Asignación de recursos según criticidad

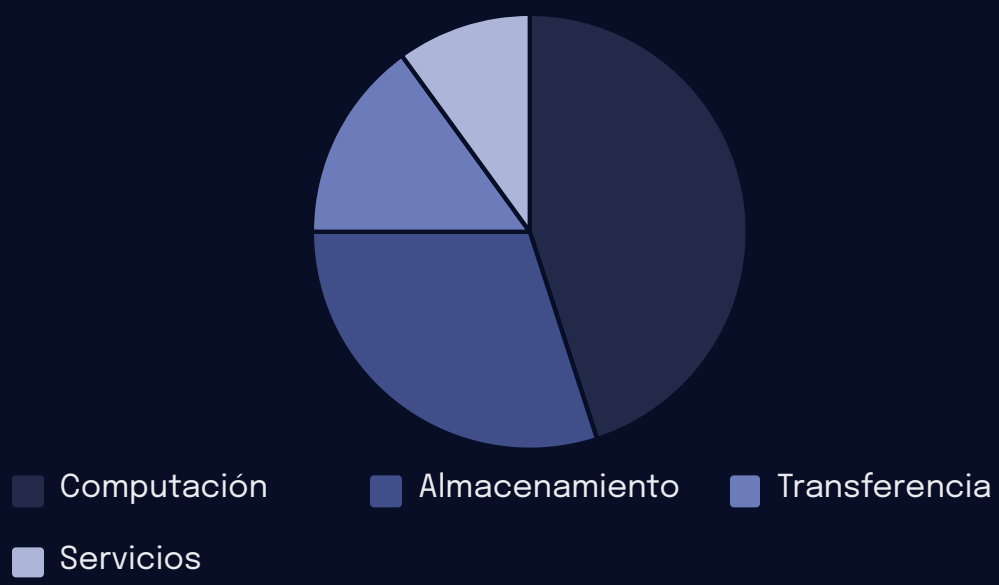
Optimización de ejecución

- **Caching**: Reutilización de resultados intermedios
- **Materialización selectiva**: Persistir solo los artefactos necesarios
- **Tamaño de lotes óptimo**: Balance entre latencia y throughput
- "**Right-sizing**": Ajuste preciso de recursos computacionales



Enfoque cost-aware desde el diseño

Un error común es optimizar pipelines solo para velocidad, sin considerar el coste. La eficiencia debe ser un requisito funcional desde el inicio.



Distribución típica de costes en pipelines de datos

SLAs, SLOs y KPIs operativos

SLA (Service Level Agreement)

Compromiso formal con el negocio sobre disponibilidad, frescura de datos o tiempo de respuesta. Suele tener implicaciones contractuales.

SLO (Service Level Objective)

Objetivo interno más estricto que el SLA. Define métricas concretas de éxito para tasa de errores, latencia o completitud.

SLI (Service Level Indicator)

Métrica real observada que se compara con el SLO. Por ejemplo: porcentaje de ejecuciones exitosas, tiempo promedio de procesamiento.

KPIs clave para orquestación

Métrica	Descripción	Objetivo típico
Tasa de éxito	% de ejecuciones sin errores	>99.5%
Duración	Tiempo de ejecución end-to-end	
Tiempo en cola	Espera antes de ejecución	<5 min
Frescura de datos	Retraso desde generación	<1 hora
Coste por ejecución	Recursos consumidos	< presupuesto asignado

Error budgets

Concepto de SRE que establece un "presupuesto de fallos" aceptable (complemento del SLO). Por ejemplo:

- Si el SLO es 99.9% de disponibilidad
- El error budget es 0.1% (43 minutos/mes)
- Mientras no se exceda, se pueden priorizar nuevas funcionalidades
- Al excederse, se priorizan fiabilidad y estabilidad

Este enfoque crea un **balance sano** entre innovación y estabilidad.

Caso de estudio: Pipeline de datos para eCommerce

1

Ingestión de datos

Captura de eventos de ventas y actualizaciones de stock mediante CDC desde la base de datos transaccional. Los datos llegan a Kafka y se procesan en micro-batches de 5 minutos.

2

Validación y limpieza

Verificación de schema, detección de duplicados y validación de business rules. Los registros sospechosos se envían a una cola de revisión mientras los válidos continúan el proceso.

3

Modelado dimensional

Transformación a modelo estrella con dimensiones (productos, clientes, tiempo) y hechos (ventas, inventario). Implementado con dbt sobre Snowflake con tests integrados.

4

Cálculo de métricas

Agregaciones para KPIs de negocio: ventas por categoría, conversión, AOV, inventario disponible. Reconciliación con sistema origen para garantizar integridad.

5

Publicación y alertas

Actualización de dashboards de BI y envío de alertas automatizadas sobre umbrales críticos (stock bajo, caídas en conversión). SLA: dashboard actualizado en menos de 15 minutos post-venta.

Lecciones aprendidas

Este caso demuestra cómo un pipeline bien diseñado combina procesamiento por eventos (ventas) con actualizaciones periódicas (agregaciones diarias), garantizando datos frescos para decisiones operativas mientras optimiza recursos computacionales.